Section Overview

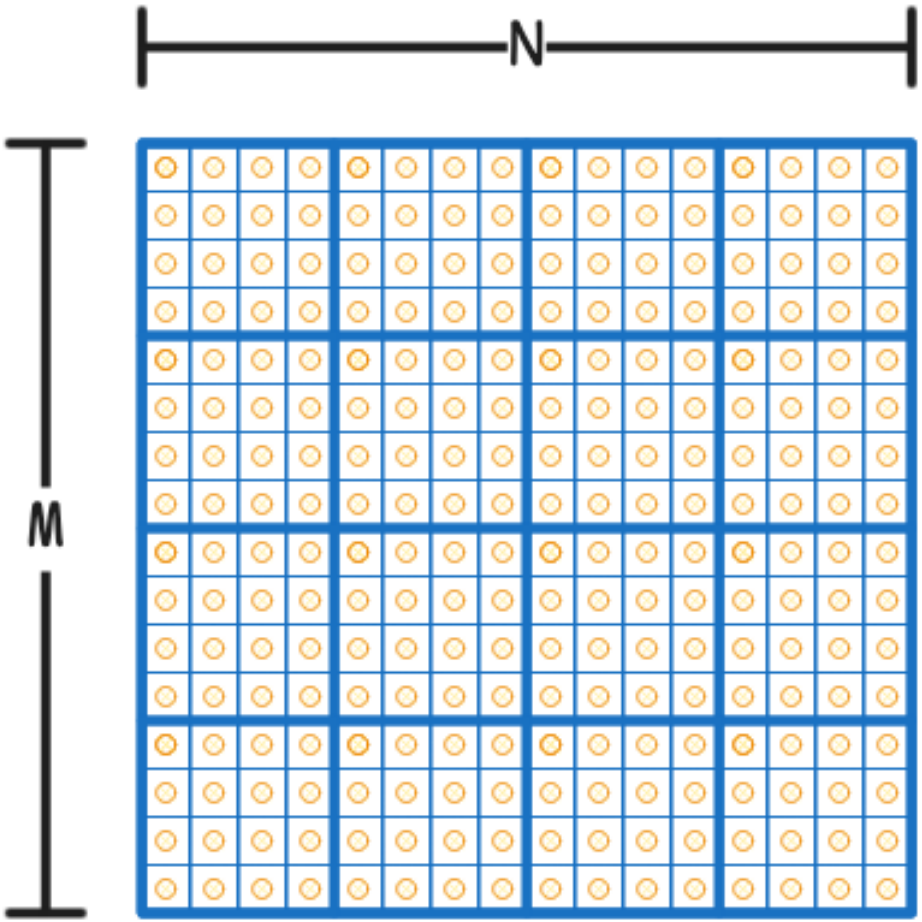
- L1 Cache and Basic Tiling

Let's do some experimenting

This kernel assigns one thread per output element, and computes them independently. So changing block size shouldn't matter right?

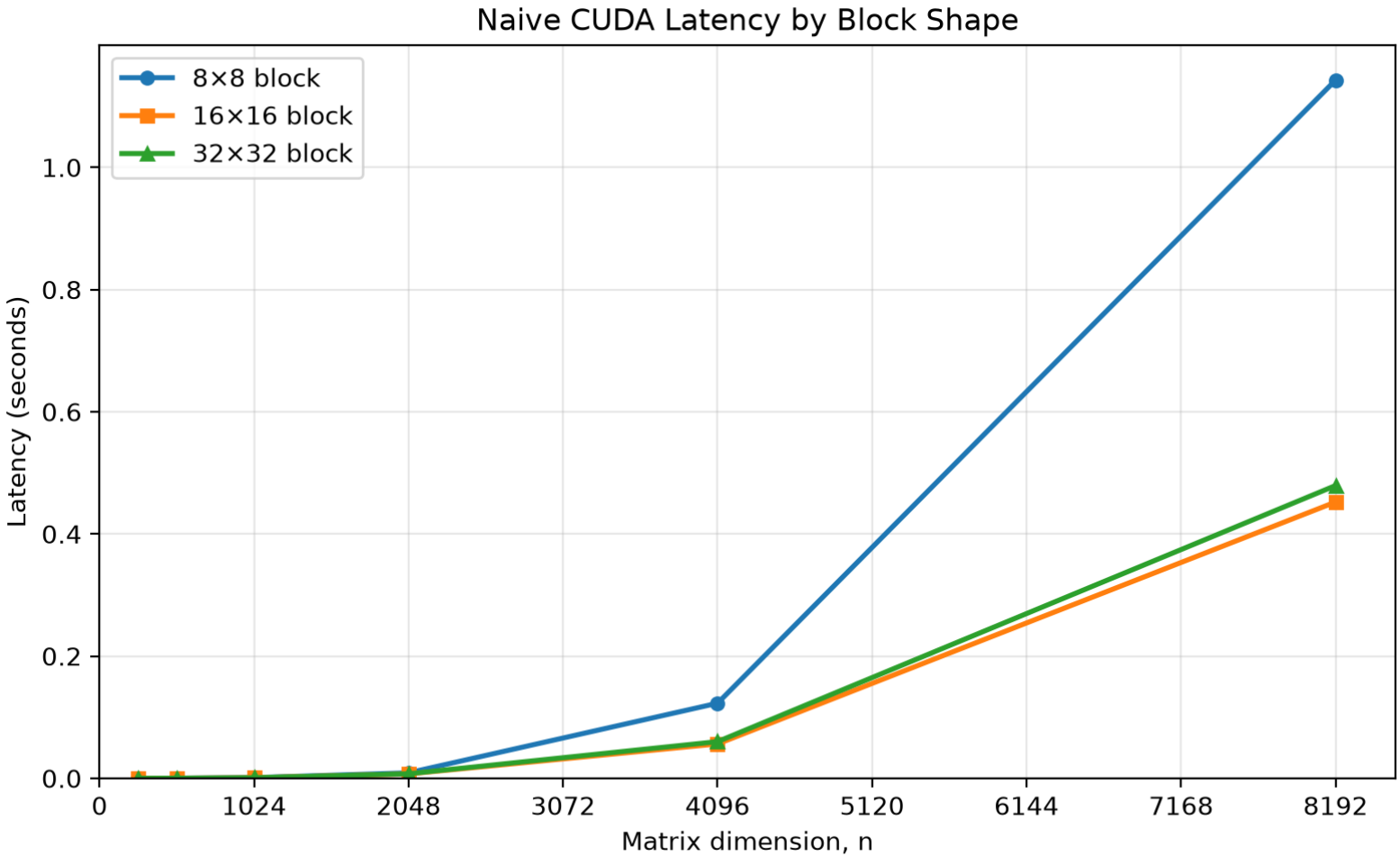
Let's do some experimenting

This kernel assigns one thread per output element, and computes them independently. So changing block size shouldn't matter right?



Let's do some experimenting

This kernel assigns one thread per output element, and computes them independently. So changing block size shouldn't matter right?



Blocks and SMs

Why does block size matter?

This happens because entire **blocks** are **arbitrarily** scheduled to SM's.
The larger block sizes of 16x16 and 32x32 lead to more **L1 cache hits**.

Blocks and SMs

Why does block size matter?

This happens because entire **blocks** are **arbitrarily** scheduled to SM's.
The larger block sizes of 16x16 and 32x32 lead to more **L1 cache hits**.

Block shape	**L1 Cache Hit Rate**
8×8	83.51%
16×16	87.13%
32×32	94.98%

RTX 3090, $M = N = K = 4096$

Blocks and SMs

Why does block size matter?

This happens because entire **blocks** are **arbitrarily** scheduled to SM's. The larger block sizes of 16x16 and 32x32 lead to more **L1 cache hits**.

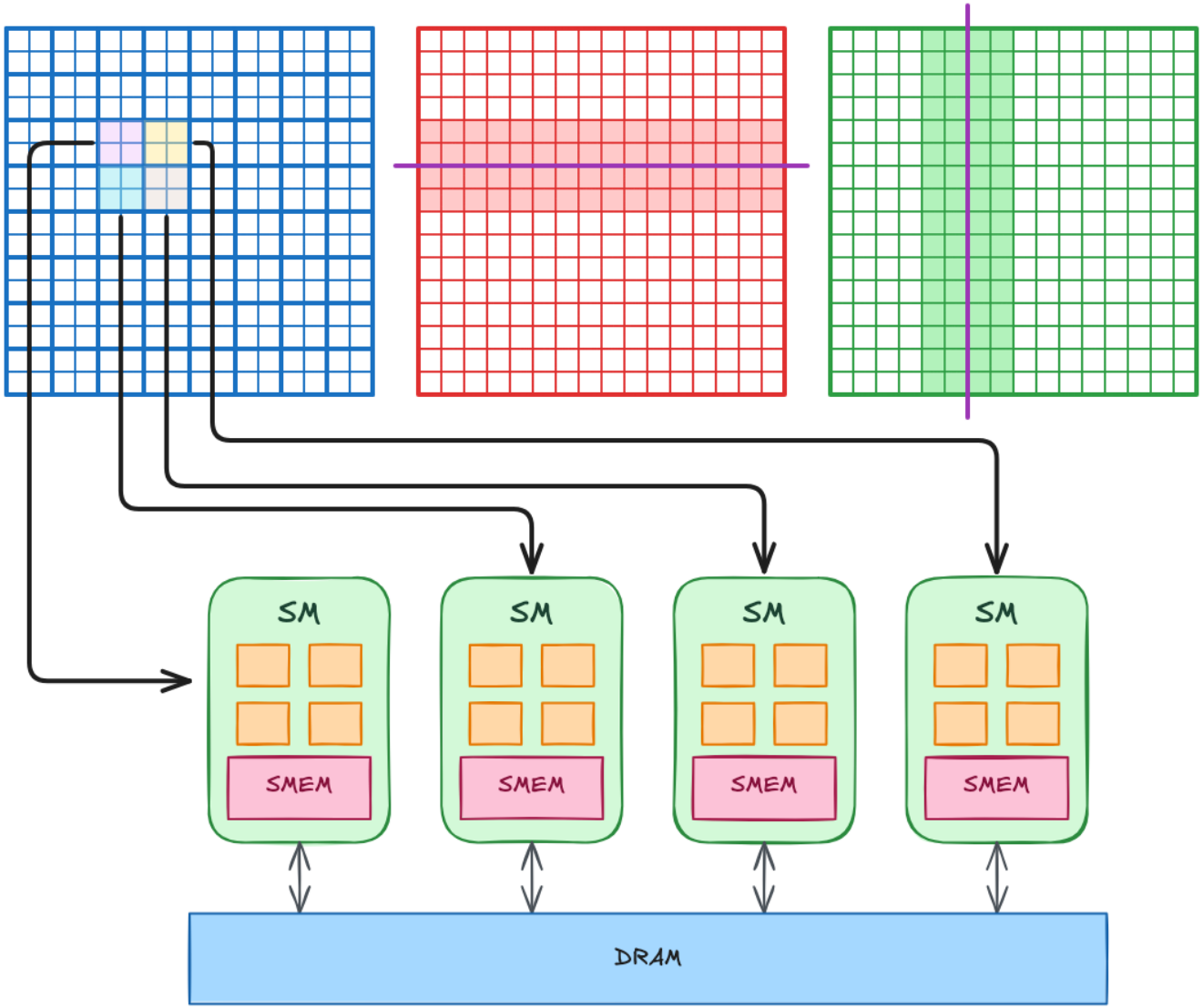
Block shape	**L1 Cache Hit Rate**
8×8	83.51%
16×16	87.13%
32×32	94.98%

RTX 3090, $M = N = K = 4096$

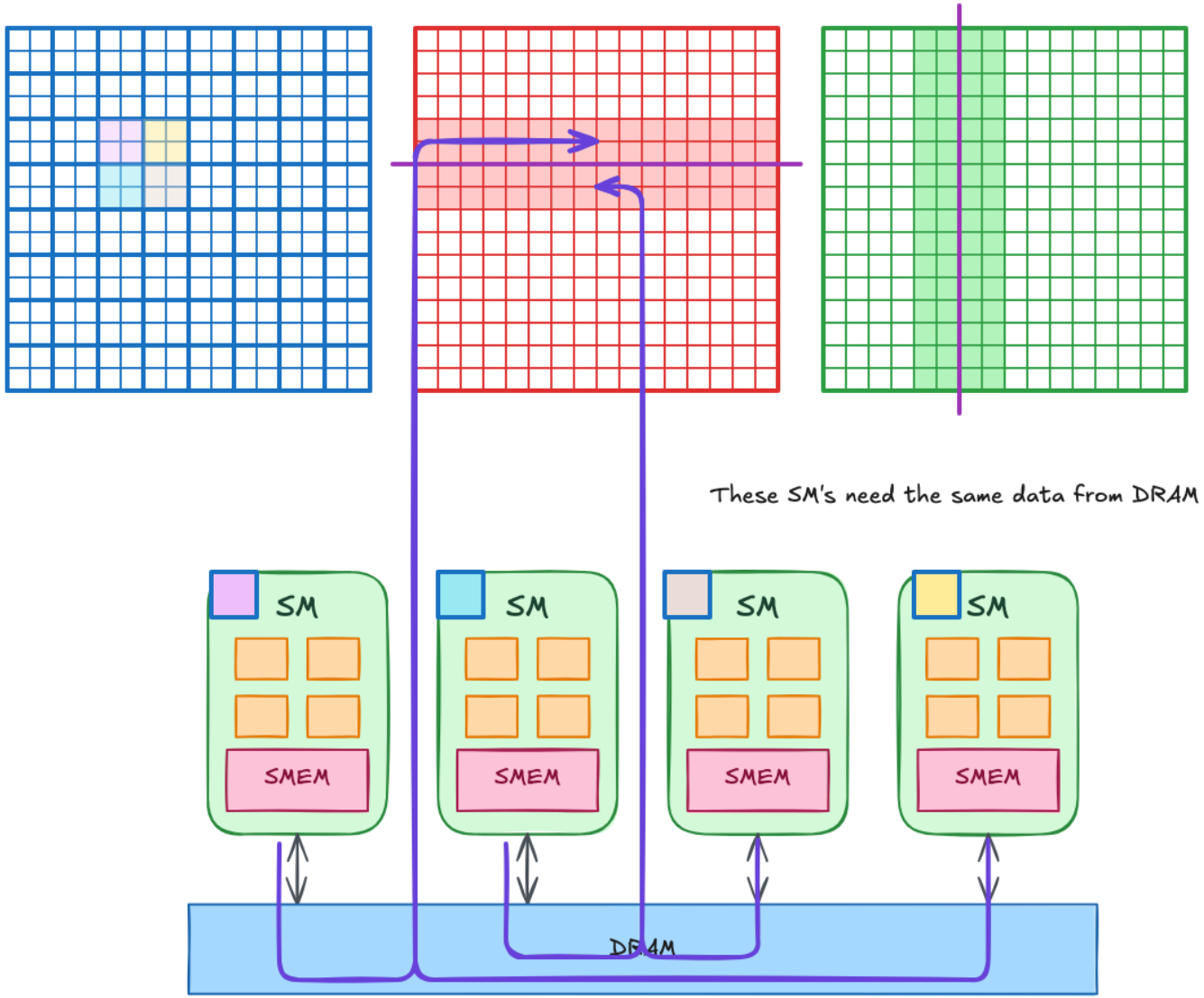
Intuition

Nearby elements of C require **nearby elements in** A **and** B. Blocks are assigned to SM's. Therefore, larger blocks mean that larger patches of C are assigned the to same SM's, increasing the chances of L1 cache hits.

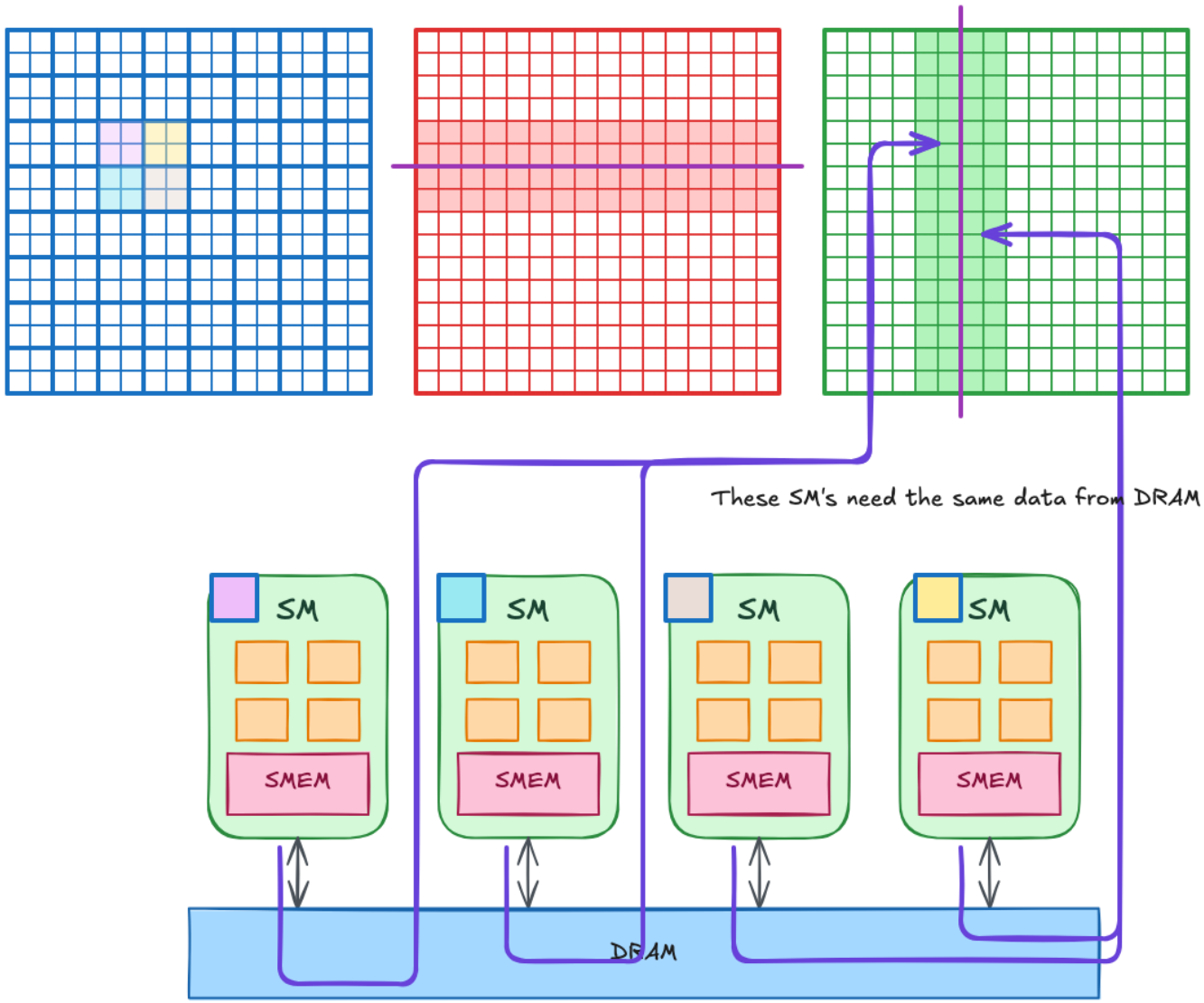
Why Larger Blocks Improve Cache Reuse



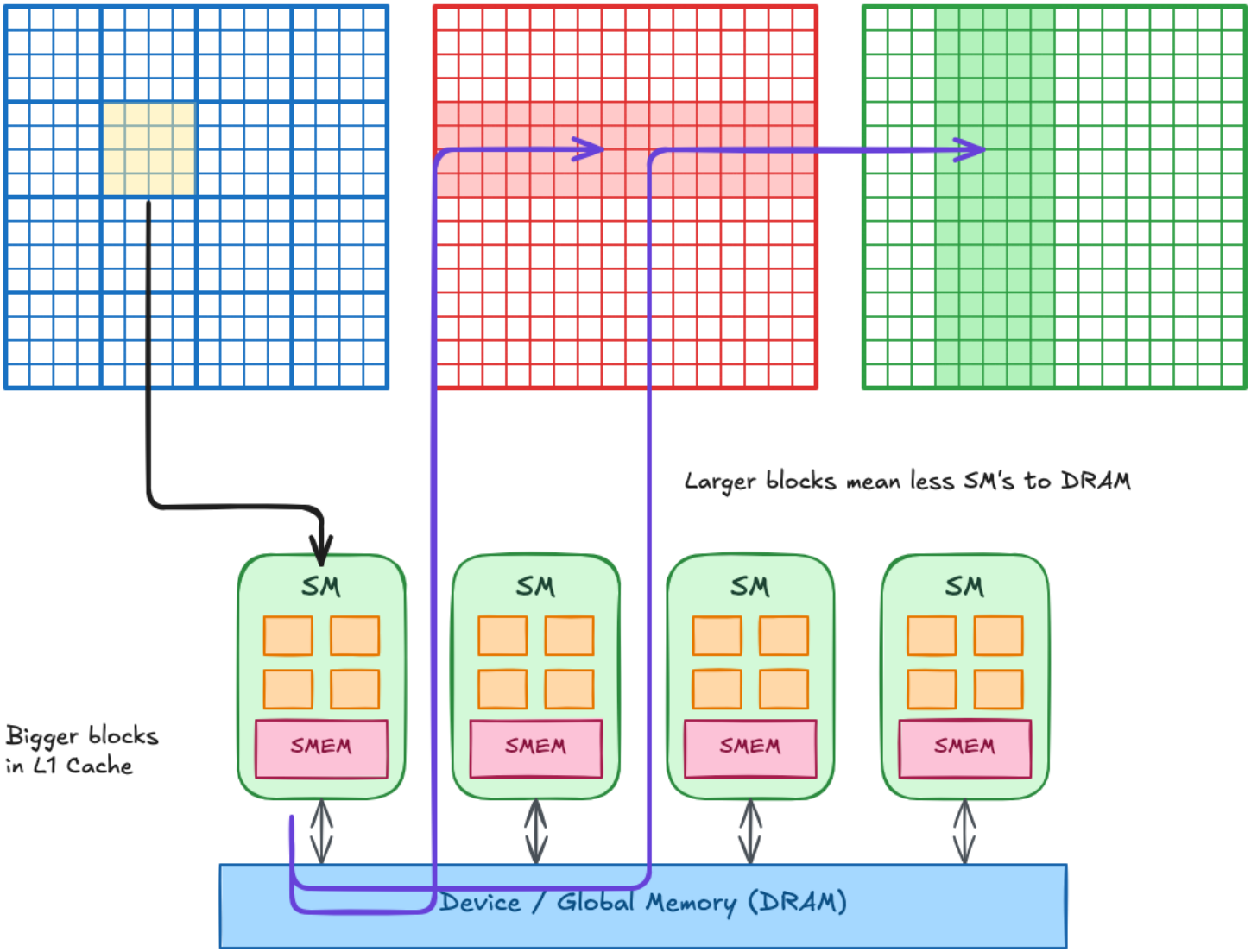
Why Larger Blocks Improve Cache Reuse



Why Larger Blocks Improve Cache Reuse



Why Larger Blocks Improve Cache Reuse



Shared Memory (SMEM) Caching

Programmable L1 Cache (SMEM)

Remember how our launch config required us to group threads as blocks? That is because **blocks** are scheduled to SM's as a unit, so **threads in the same block have access to the SM's L1 cache/SMEM.**

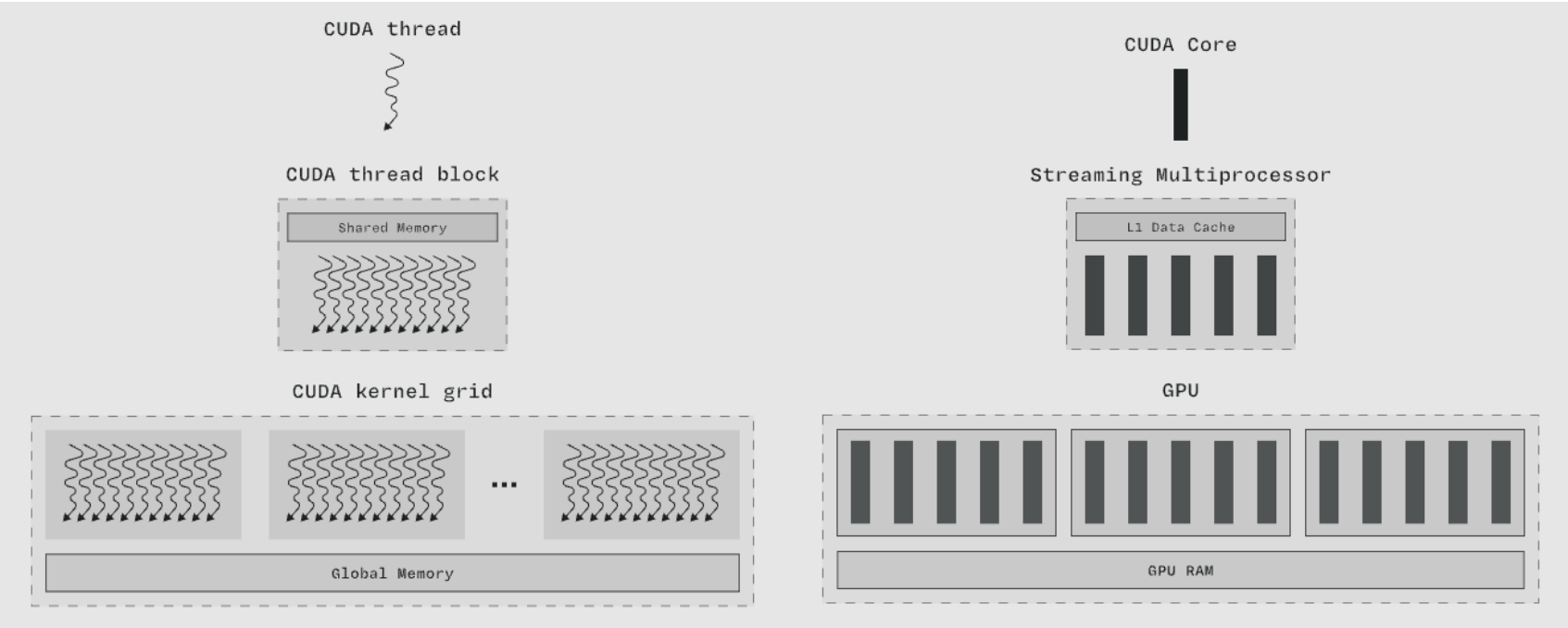


Image source: Modal, "CUDA Programming Model," *GPU Glossary*.

Tiling

Explicit caching of data shared between threads.

Main Idea

Before doing any computation, load a chunk of the necessary data from DRAM to SMEM as a block. Then, memory accesses for the computation now only go to SMEM.

Basic tiled SGEMM: step by step

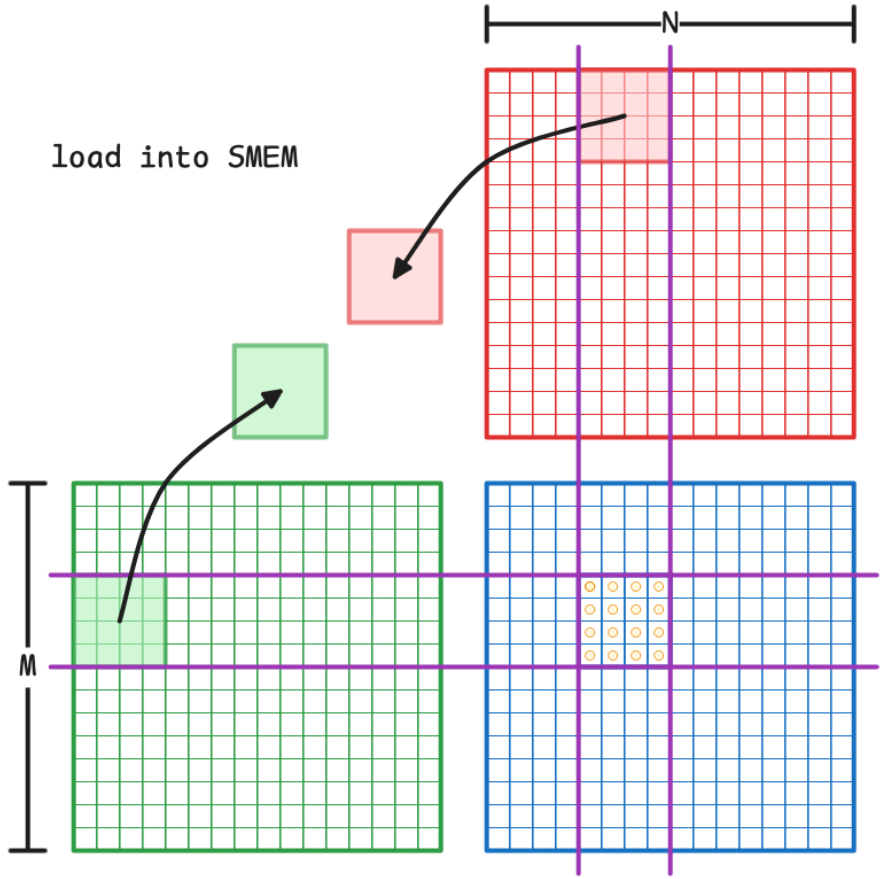
```
__shared__ float As[BK][BK];
__shared__ float Bs[BK][BK];
float sum = 0.0;

for (int k = 0; k < K; k += BK) {

  const int A_tile_x = k + xt;
  if (y < M && A_tile_x < K) {
    As[yt][xt] = A[y * K + A_tile_x];
  } else {
    As[yt][xt] = 0.0;
  }

  const int B_tile_y = k + yt;
  if (B_tile_y < K && x < N) {
    Bs[yt][xt] = B[B_tile_y * N + x];
  } else {
    Bs[yt][xt] = 0.0;
  }
  __syncthreads();

  for (int i = 0; i < BK; i++) {
    sum += As[yt][i] * Bs[i][xt];
  }
  __syncthreads();
}
```



Basic tiled SGEMM: step by step

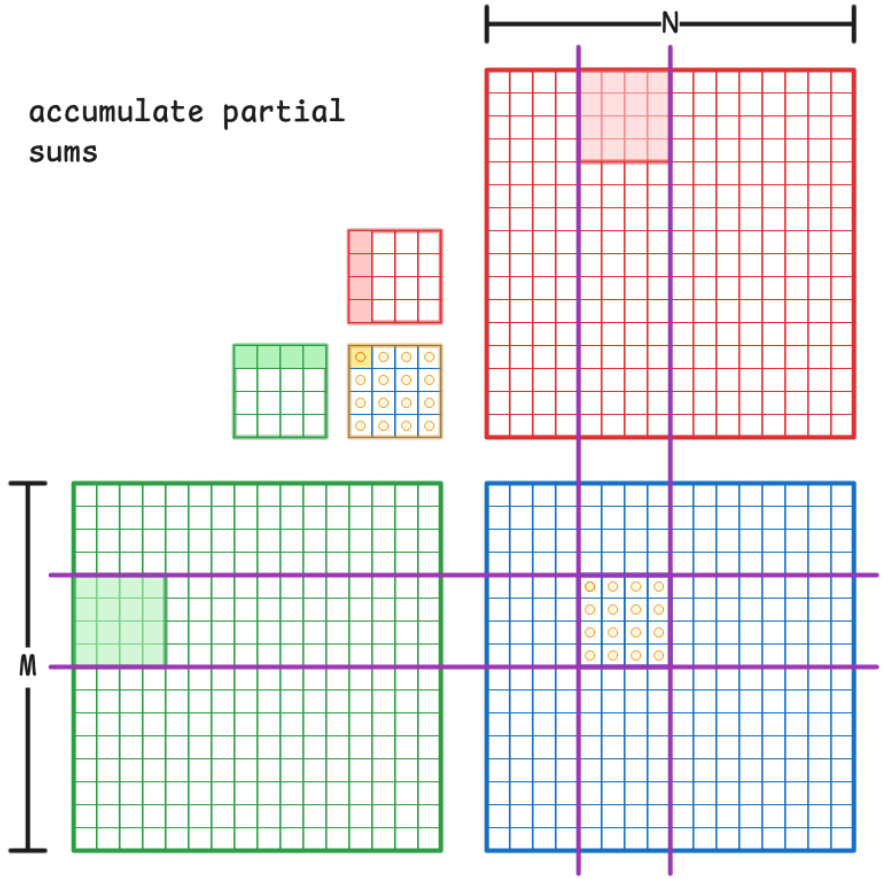
```
__shared__ float As[BK][BK];
__shared__ float Bs[BK][BK];
float sum = 0.0;

for (int k = 0; k < K; k += BK) {

  const int A_tile_x = k + xt;
  if (y < M && A_tile_x < K) {
    As[yt][xt] = A[y * K + A_tile_x];
  } else {
    As[yt][xt] = 0.0;
  }

  const int B_tile_y = k + yt;
  if (B_tile_y < K && x < N) {
    Bs[yt][xt] = B[B_tile_y * N + x];
  } else {
    Bs[yt][xt] = 0.0;
  }
  __syncthreads();

  for (int i = 0; i < BK; i++) {
    sum += As[yt][i] * Bs[i][xt];
  }
  __syncthreads();
}
```



Basic tiled SGEMM: step by step

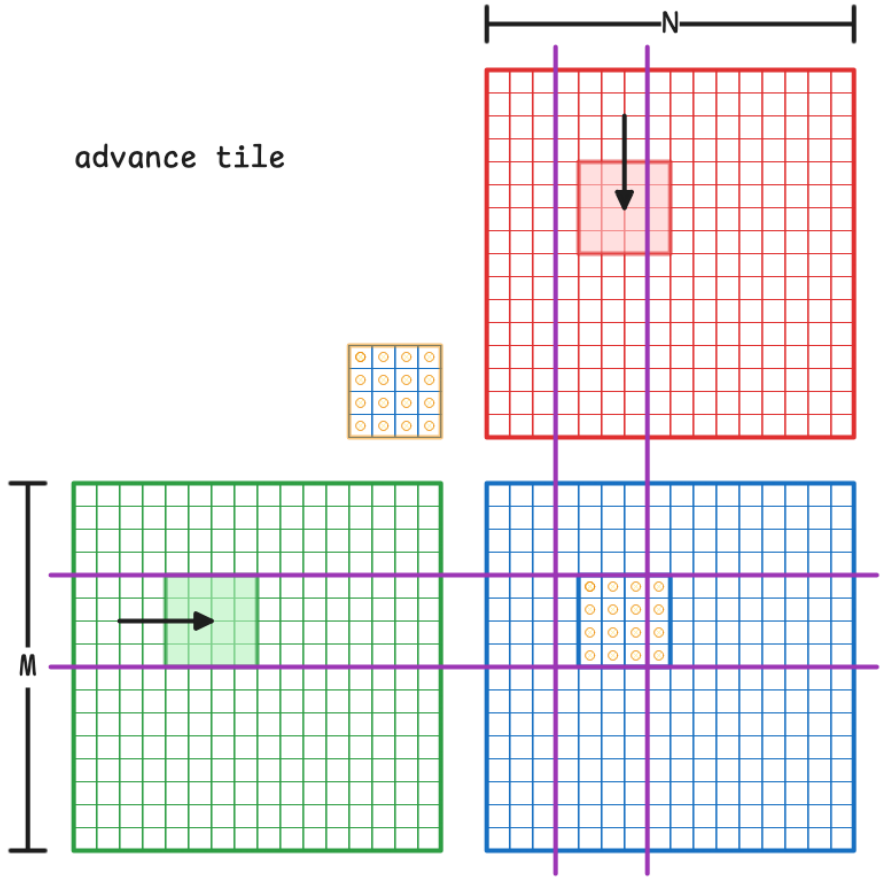
```
__shared__ float As[BK][BK];
__shared__ float Bs[BK][BK];
float sum = 0.0;

for (int k = 0; k < K; k += BK) {

  const int A_tile_x = k + xt;
  if (y < M && A_tile_x < K) {
    As[yt][xt] = A[y * K + A_tile_x];
  } else {
    As[yt][xt] = 0.0;
  }

  const int B_tile_y = k + yt;
  if (B_tile_y < K && x < N) {
    Bs[yt][xt] = B[B_tile_y * N + x];
  } else {
    Bs[yt][xt] = 0.0;
  }
  __syncthreads();

  for (int i = 0; i < BK; i++) {
    sum += As[yt][i] * Bs[i][xt];
  }
  __syncthreads();
}
```



Basic tiled SGEMM: step by step

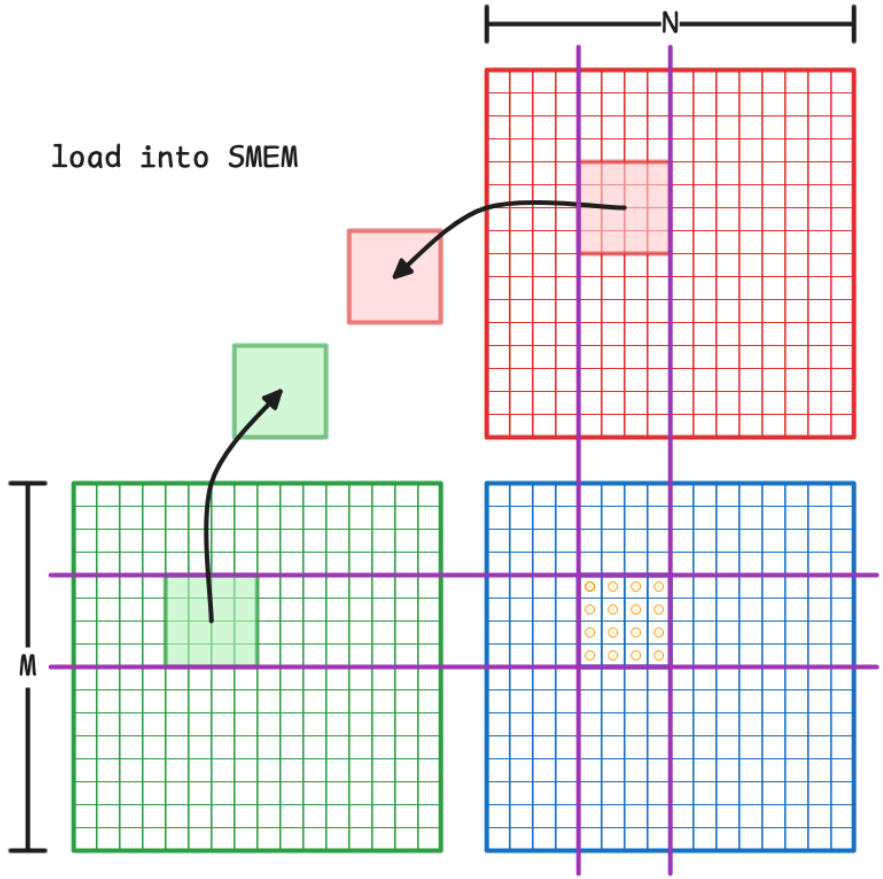
```
__shared__ float As[BK][BK];
__shared__ float Bs[BK][BK];
float sum = 0.0;

for (int k = 0; k < K; k += BK) {

  const int A_tile_x = k + xt;
  if (y < M && A_tile_x < K) {
    As[yt][xt] = A[y * K + A_tile_x];
  } else {
    As[yt][xt] = 0.0;
  }

  const int B_tile_y = k + yt;
  if (B_tile_y < K && x < N) {
    Bs[yt][xt] = B[B_tile_y * N + x];
  } else {
    Bs[yt][xt] = 0.0;
  }
  __syncthreads();

  for (int i = 0; i < BK; i++) {
    sum += As[yt][i] * Bs[i][xt];
  }
  __syncthreads();
}
```



Basic tiled SGEMM: step by step

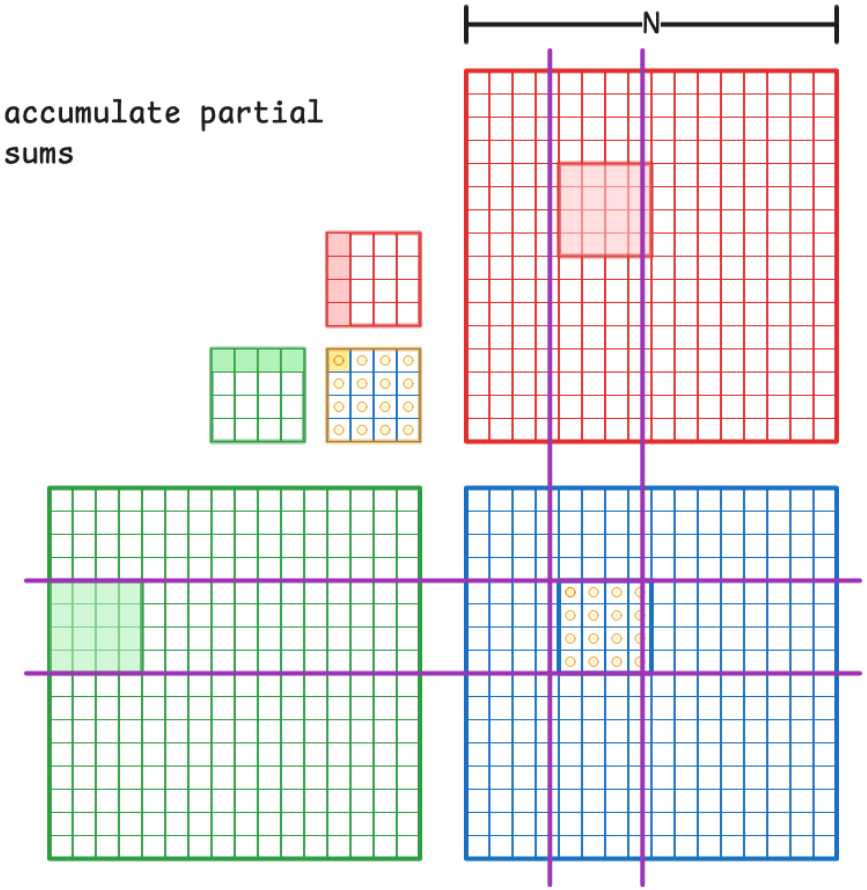
```
__shared__ float As[BK][BK];
__shared__ float Bs[BK][BK];
float sum = 0.0;

for (int k = 0; k < K; k += BK) {

  const int A_tile_x = k + xt;
  if (y < M && A_tile_x < K) {
    As[yt][xt] = A[y * K + A_tile_x];
  } else {
    As[yt][xt] = 0.0;
  }

  const int B_tile_y = k + yt;
  if (B_tile_y < K && x < N) {
    Bs[yt][xt] = B[B_tile_y * N + x];
  } else {
    Bs[yt][xt] = 0.0;
  }
  __syncthreads();

  for (int i = 0; i < BK; i++) {
    sum += As[yt][i] * Bs[i][xt];
  }
  __syncthreads();
}
```



Basic tiled SGEMM: step by step

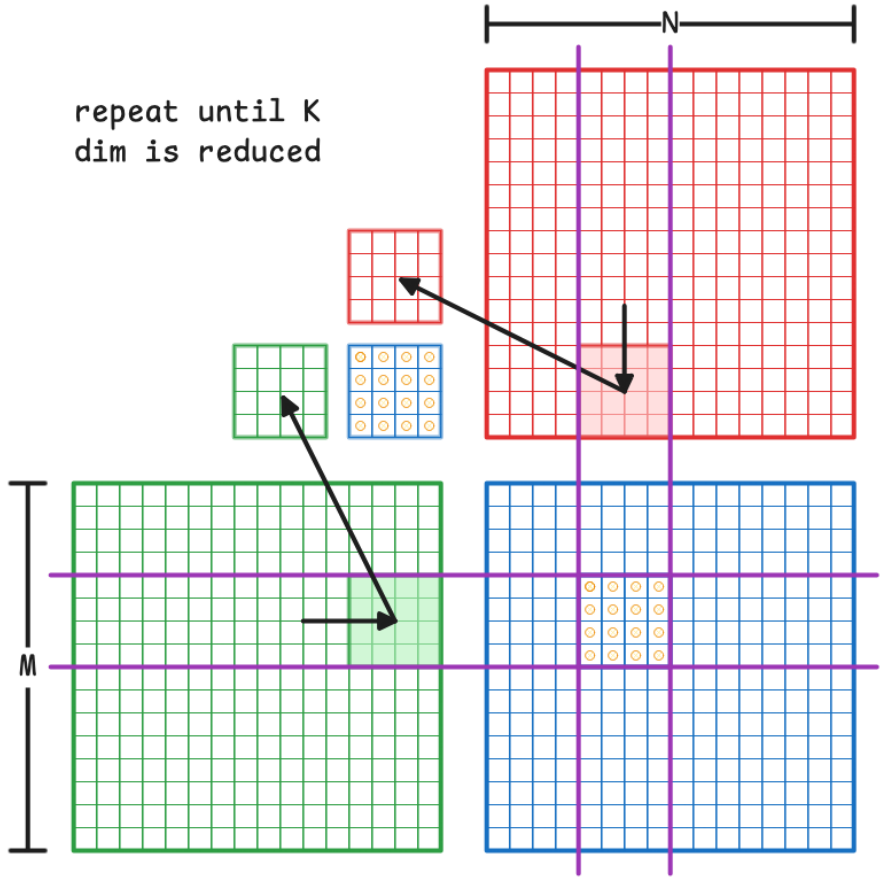
```
__shared__ float As[BK][BK];
__shared__ float Bs[BK][BK];
float sum = 0.0;

for (int k = 0; k < K; k += BK) {

  const int A_tile_x = k + xt;
  if (y < M && A_tile_x < K) {
    As[yt][xt] = A[y * K + A_tile_x];
  } else {
    As[yt][xt] = 0.0;
  }

  const int B_tile_y = k + yt;
  if (B_tile_y < K && x < N) {
    Bs[yt][xt] = B[B_tile_y * N + x];
  } else {
    Bs[yt][xt] = 0.0;
  }
  __syncthreads();

  for (int i = 0; i < BK; i++) {
    sum += As[yt][i] * Bs[i][xt];
  }
  __syncthreads();
}
```

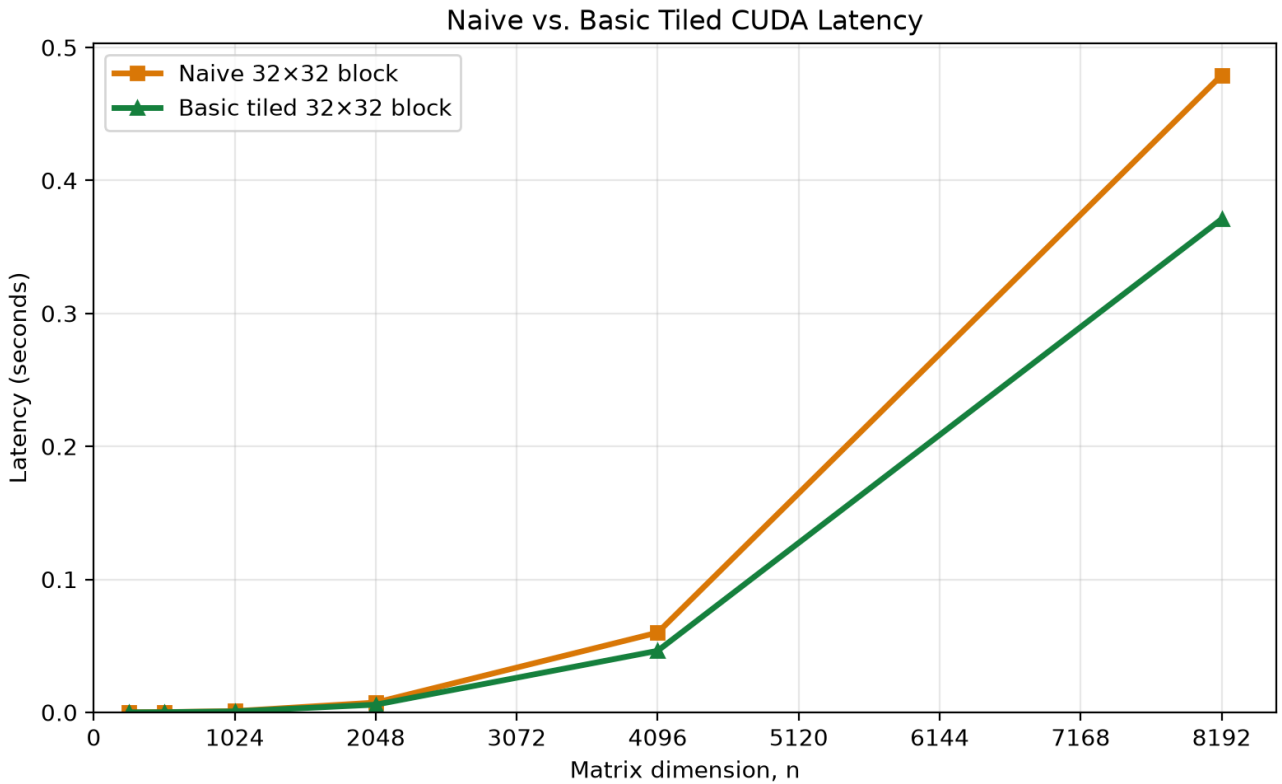


Basic tiled SGEMM visualization

- Watch each block load small regions of A and B together.
- The loaded tiles are reused to update an entire tile of C.
- Synchronization separates loading, computing, and advancing to the next tiles.

Play tiled-kernel visualization

Tiling performance



At $n, m, k = 8192$, tiled 32×32 latency is 371 ms, a 22.6% improvement over the naive.